

DSD LAB FINAL PREPARATION

Topic:01 [ALU]

1. Arithmetic logic unit (ALU) is a digital circuit used to perform arithmetic and logic operations.
2. How Does an ALU work?
 - Most of the operations of a CPU are performed by one or more ALUs, which load data from input registers.
 - **A register is a small amount of storage available as part of a CPU.**
 - The control unit tells the ALU what operation to perform on that data and the ALU stores the result in an output register.
 - The control unit moves the data between these registers, the ALU, and memory.
3. Operations in ALU

Arithmetic

 - Addition ($F = A + B$)
 - Addition with Carry ($F = A + B + 1$)
 - Subtraction ($F = A - B$ or $F = A + B' + 1$)
 - Subtraction with Borrow ($F = A - B - 1$ or $F = A + B'$)
 - Increment ($F = A + 1$)
 - Decrement ($F = A - 1$)
 - Transfer ($F = A$)
 - Transfer with Carry ($F = A ; Cout = 1$)

Logical

 - And ($F = A . B$)
 - Or ($F = A + B$)
 - XOR ($F = A \text{ xor } B$)
 - NOT ($F = A'$)
4. 4-bit ALU

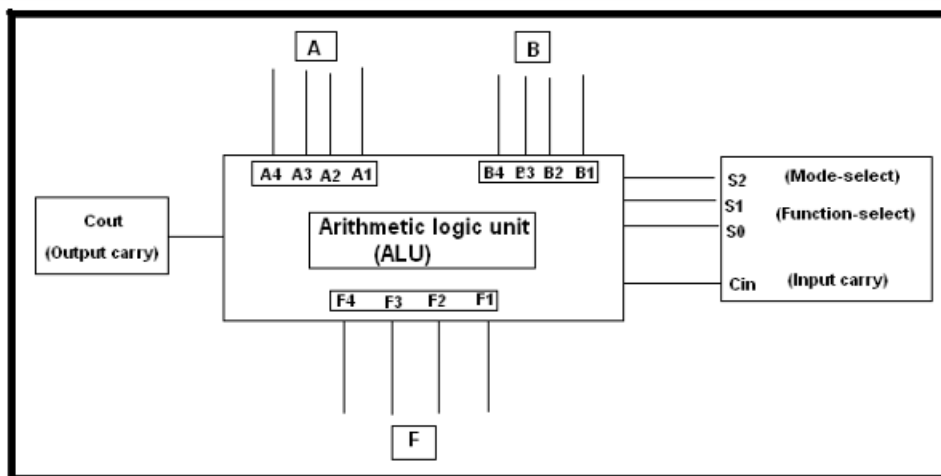
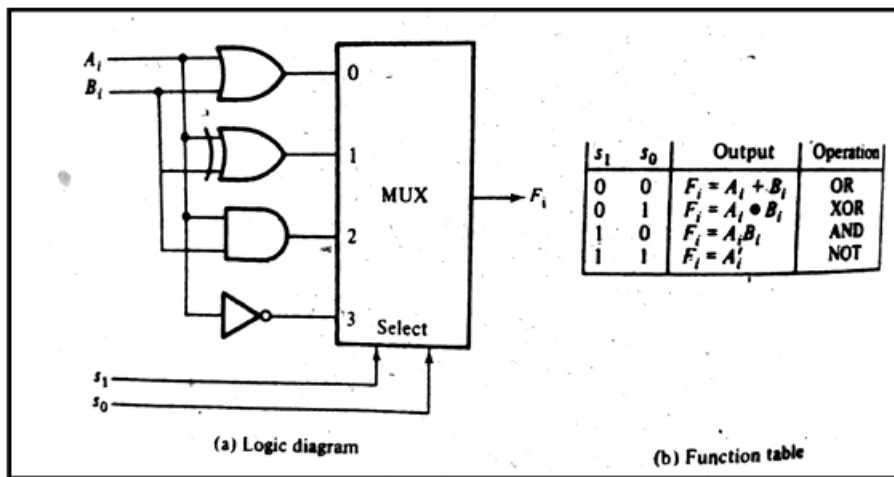


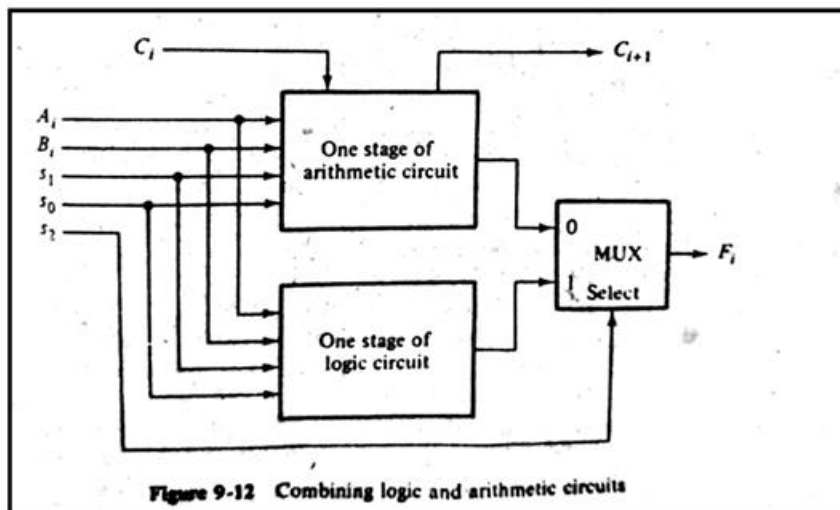
Fig: Block Diagram of a 4-bit ALU

5. Design of Logic Circuit



The circuit must be repeated n times for an n-bit logic circuit

6. Combining Logic and Arithmetic Circuits



Disadvantages :

- Requires a lot of ICs
- Need to build both the arithmetic and logic circuit distinctively
- More processing time with more overhead

7. A More Efficient ALU (Contd.)

- Easier to generate logic operations in an already available arithmetic circuit
- Cinalways 0 (or, don't care) when selection variable $S_2=1$
- When $S_2=1$, combination of S_1 and S_0 selects the 4 logical operations
- When $S_2=0$, combination of S_1, S_0 and C_{in} selects the 8 arithmetic operations

Steps:

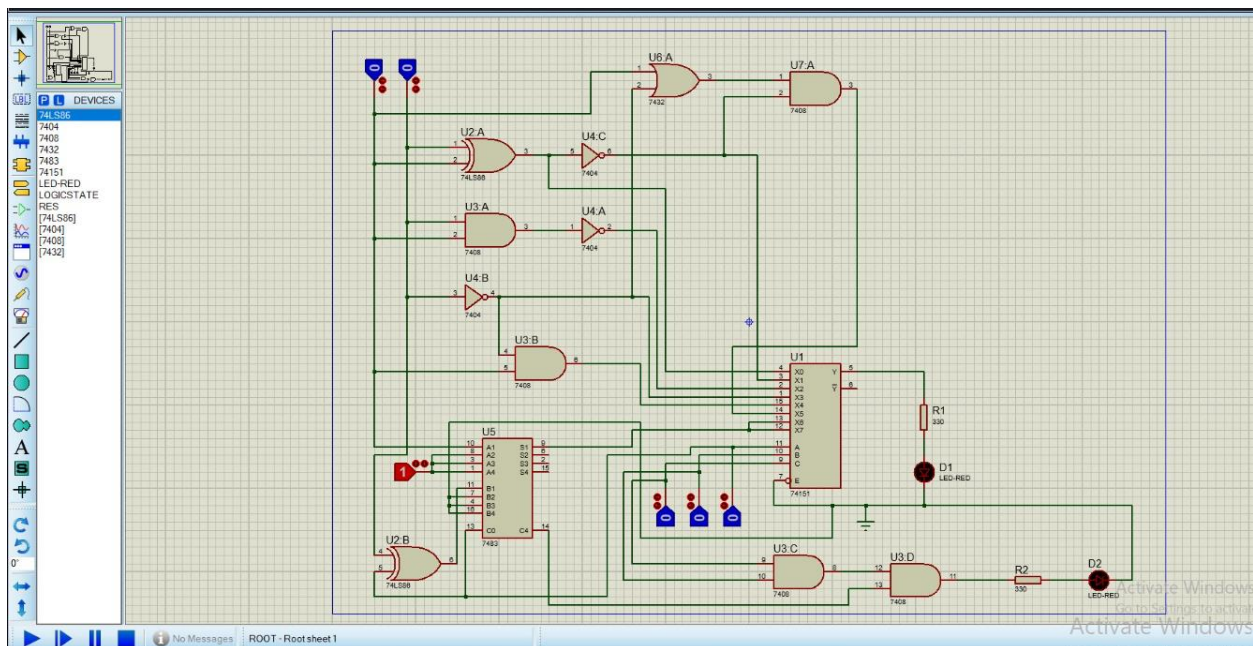
1. Design the arithmetic section independent of the logic section
2. Determine the logic operations obtained from the arithmetic circuit in step 1, assuming that the input carries to all stages are 0
3. Modify the arithmetic circuit to obtain the required logic operations

8. Important parts

$$1111 = \text{all } 1 = -1$$

$$F \leq 2^n - 1, \text{ Count} = 0$$

$$F \geq 2^n, \text{ Count} = 1$$



(৪ই)
Effects of Output Carry এর চক্ৰতা:

$$\begin{array}{r}
 1111 \\
 \underline{1} \\
 (1)0000
 \end{array}
 \begin{array}{l}
 \rightarrow 2^n - 1 \\
 \hline
 1 \\
 2^n
 \end{array}$$

$$C_{out} = 1 \text{ if } F \geq 2^n$$

① $F = A$ transfer A with $C_{out} = 0$

$$\begin{array}{ccc}
 F = A + 0 + 0 \\
 \downarrow \quad \downarrow \quad \downarrow \\
 X \quad Y \quad C_{in}
 \end{array}$$

A 4 bit এর।
 maximum যোগ্যে
 পারবে 1111, so C_{out}
 যোগ্যো possible না।

C_{out} will always
 be 0.

② $F = A + 1$, increment

$$C_{out} = 1 \text{ if } F \geq 2^n$$

$$\Rightarrow A + 1 \geq 2^n$$

$$\Rightarrow A \geq 2^n - 1$$

③ $F = A + B$, Addition

$$C_{out} = 1 \text{ if } F \geq 2^n$$

$$\Rightarrow A + B \geq 2^n$$

$$(4) F = A + B + 1$$

$$Count = 1 \text{ if } \cancel{F = A + B + 1} \quad F \geq 2^n$$

$$\Rightarrow A + B + 1 \geq 2^n$$

$$\Rightarrow A + B \geq 2^n - 1$$

$$(5) F = A - B - 1 = A + \bar{B}$$

$$Count = 1 \text{ if } F \geq 2^n$$

$$\Rightarrow A + \bar{B} \geq 2^n$$

$$\Rightarrow A + 2^n - 1 - B \geq 2^n$$

$$\Rightarrow A - B - 1 \geq 0$$

$$\Rightarrow A \geq B + 1$$

$$\Rightarrow A > B$$

$$-B = \bar{B} + 1$$

$$\Rightarrow \bar{B} = -B - 1$$

$$= -1 - B$$

$$= \text{All } 1$$

$$-B$$

$$= 2^n - 1 - B$$

$$\text{All } 1 = \overline{1111}$$

$$= 15$$

$$= 2^n - 1$$

$$A \geq 8$$

$$A > 7$$

A, 8 এর সমান/বড় অর্থাৎ A
অবশ্যই 7 এর বড়,

সুপ্রাইম

$$\textcircled{6} \quad F = A - B = A + \bar{B} + 1 \quad \bar{B} = 2^n - 1 - B$$

$$\begin{aligned} \text{Cont} = 1 \quad \text{if} \quad F > 2^n \\ \Rightarrow A + \bar{B} + 1 > 2^n \\ \Rightarrow A + 2^n - 1 - B + 1 > 2^n \\ \Rightarrow A - B > 0 \\ \therefore A > B \end{aligned}$$

$$\textcircled{7} \quad F = A - 1 \quad -1 = \text{All } 1$$

$$\begin{aligned} \text{Cont} = 1 \quad \text{if} \quad F > 2^n \\ \Rightarrow A - 1 > 2^n \\ \Rightarrow A + \text{All } 1 > 2^n \\ \Rightarrow A + 2^n - 1 > 2^n \\ \Rightarrow A - 1 > 0 \\ \Rightarrow A > 1 \\ \Rightarrow A > 0 \end{aligned}$$

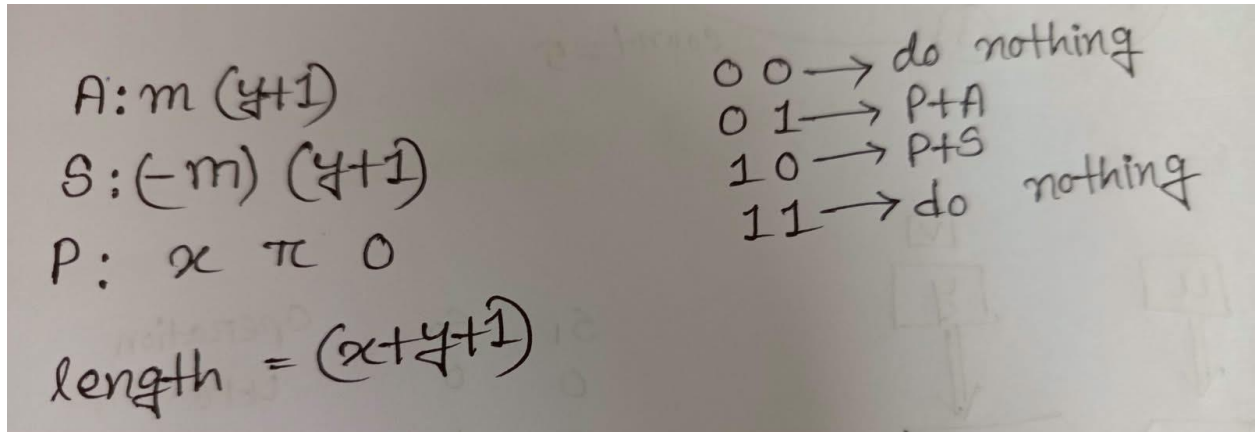
$$\textcircled{8} \quad F = A, \text{Cont} = 1$$

$$\begin{aligned} F &= A + \text{All } 1 + 1 \\ &= A + 2^n - 1 + 1 \\ &= A + 2^n \end{aligned}$$

Cont will always be 1

Topic-02 [Booth]

1. Booth's multiplication algorithm can be implemented by repeatedly adding one of two predetermined values A and S to a product P, then performing a rightward arithmetic shift on P. Let m and r be the multiplicand and multiplier respectively. And let x and y represent the number of bits in m and r.
2. Steps



A: Fill the most significant (left most) bits with the value of m. Fill the remaining $(y + 1)$ bits with zeros.

S: Fill the most significant bits with the value of $(-m)$ in 2's complement notation. Fill the remaining $(y + 1)$ bits with zeros.

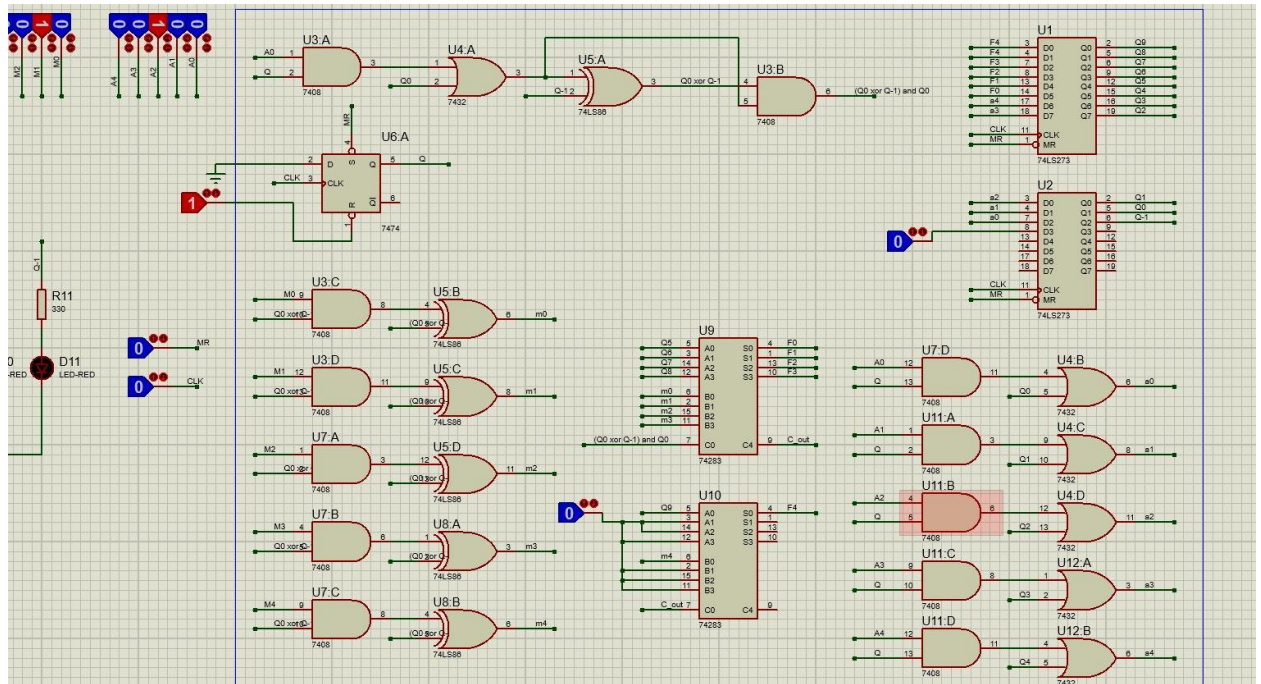
P: Fill the most significant x bits with zeros. To the right of this, append the value of r. Fill the least significant (right most) bit with a zero.

Arithmetically shift the value obtained in the second step by a single place to the right. Now, let P equals this new value.

Repeat steps 2 and 3 until they have been done y times.

Drop the least significant (right most) bit from P.

3. Proteus



Topic-03[RAM]

RAM is the place in a computer where the operating system , application programs, and data in current use are kept so that they can be quickly reached by the computer's processor.

Memory unit: Stores binary information in groups of bits called words

Dynamic RAM (DRAM)	Static RAM (SRAM)
① Reduced power consumption	① high power consumption
② high density	② low density
③ Cheap	③ Expensive.
④ Slower	④ Faster

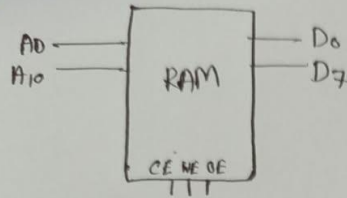
③ To transfer a new word to be stored into memory:

- ① Apply the binary address of the word to address lines.
- ② Apply the data bits that must be stored in memory to the data input lines.
- ③ Activate the write input.

④ To transfer a stored word out of memory:

- ① Apply the binary address of the word to address lines.
- ② Activate the read input.

RAM



Address $\Rightarrow$ 11

Data $\Rightarrow$ 8

Write $\Rightarrow$ Ram କ୍ରମିତ
data ଗ୍ରହଣ

Read $\Rightarrow$ Ram କ୍ରମିତ
data ଗ୍ରହଣ

data store
WE = 0 $\rightarrow$ enable
WE = 1 $\rightarrow$ disable

active low
CE/CS $\rightarrow$ chip enable
WE $\rightarrow$ write "
OE $\rightarrow$ output "

CS/CE = 0 $\rightarrow$ selected/active
CS/CE = 1 $\rightarrow$ inactive

OE = 0 $\rightarrow$ enable
OE = 1 $\rightarrow$ disable } ଫିଡ୍‌ବ୍ୟାକ୍ data
ଅନୁପ୍ରାପ୍ତ

use toggle switches for input
ଅଥବା 0/1 input ଲଗା

outside $\rightarrow$ D0 ... D7 $\rightarrow$ RAM
RAM $\rightarrow$ D0 ... D7 $\rightarrow$ outside } Bi-directional

A1 ques

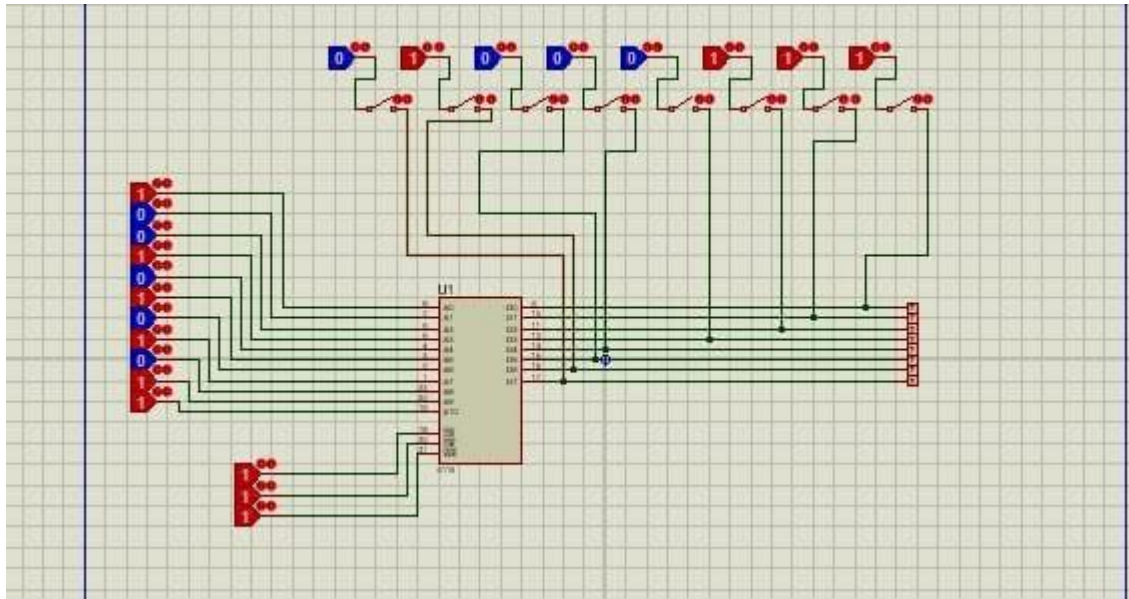
Using a 6116 SRAM, input and output the first letter of each group member's hometown or birth city. The specific memory address (x) for each member's letter will be the last two digits of their respective mobile phone number. Use toggle switches for your input and LEDs for output.

B1 ques

Using a 6116 RAM in Proteus, input and output the first letter of each group member's mother's name. The starting memory address for each group's data will be the sum of the digits of the youngest member's birth year & the next 5 places. Use toggle switches [for] input.

A2 ques

Proteus



Topic-04[SAP-1]

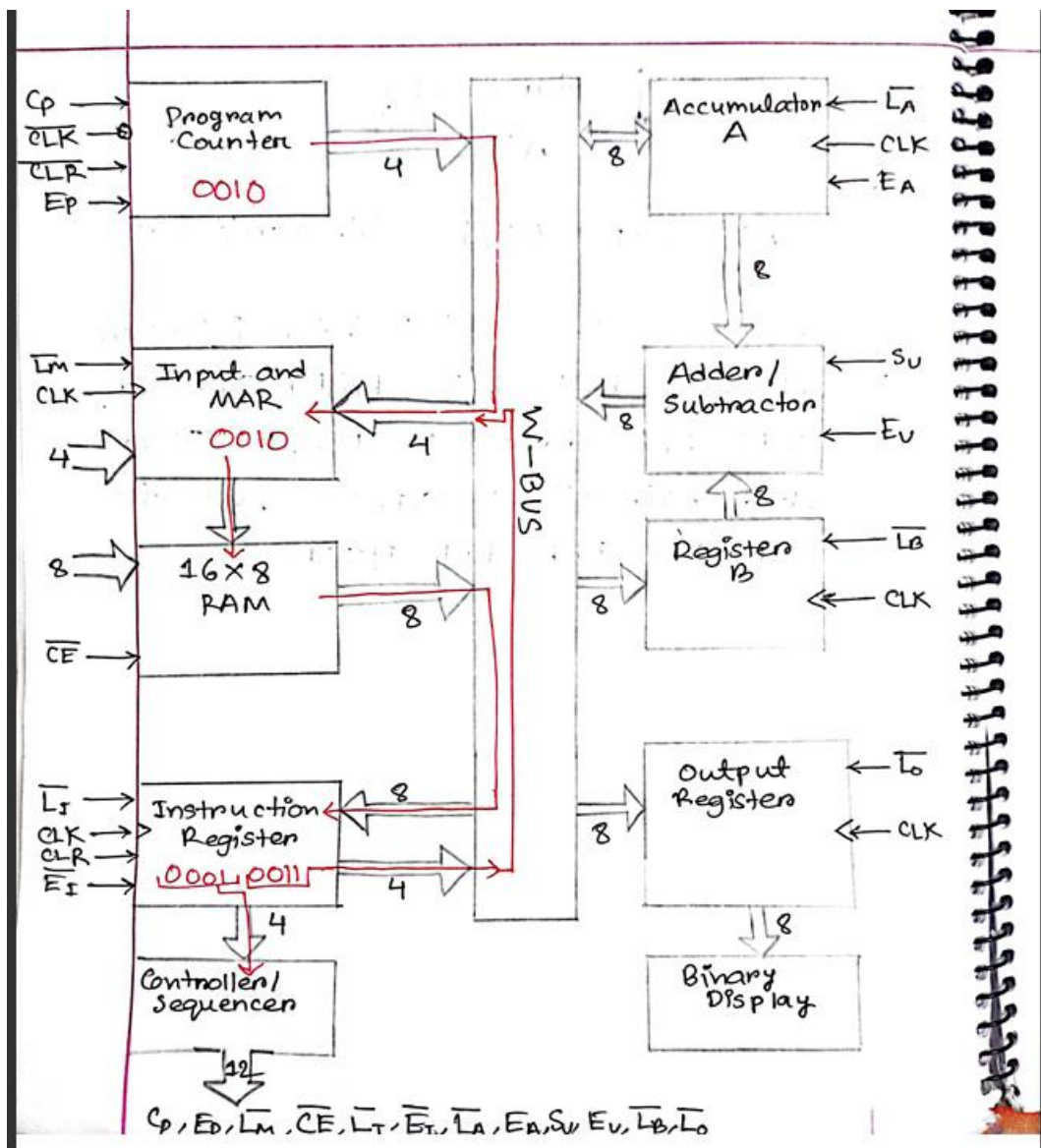
Lecture - 11

SAP-1

SAP = Simple As possible Computer

Program Counter : Pointer Register

এর মতো করে কাজ করে। যেখানে
আমি ছাই address তা রাখবে, জানে
কোন instruction তা পরবর্তীতে execute
হবে সেটা চিহ্ন করে রাখবে।



Program Counter থেকে একটা address যায় MAR এ। এর কাজ হচ্ছে store করে রাখা temporary. Store করে যে পাঠিয়ে দেয় RAM এ, RAM এ গিয়ে ঐ address টা খুঁজতে হয়। Address এ কি value আছে খুঁজতে হয়। যেই 8-bit এর data টাকে W-Bus এ পাঠিয়ে দেয়। যেখান থেকে যেটা চলে আসে instruction register এ। ২টা ভাগে ভাগ হয়। ১টা ভাগ চলে যায় controller এ, আরেকটা W-Bus এ। controller এ যেটা যায় যেটাতে একটা instruction থাকে। instruction অনুযায়ী পাঠিয়ে দেয় 12-bit এর control signal. আরেকটা ভাগ (4-bit) যেটা W-Bus এ চলে গিয়েছিলো যেটা আবার MAR এ যায়। যেখান থেকে আবার RAM এ গিয়ে যেই value টা পাচ্ছে, যদি ADD হয়ে থাকে, (ADD 3H) 3H যেই value টা আছে address

RAM এর, যেই value তা আছে, সেটা bit এ বেখে add করতে হবে

accumulator এর value এর সাথে।

3H value তা RAM থেকে B Register এ load করতে হবে, Load করতে হলে $\overline{L_B} = 0$ দিতে হবে।

এরপর দুইটা যোগ হবে Accumulator এর 8-bit এর value এবং Register B এর 8-bit এর value। দুইটা যোগ হয়ে যোগফল তা চলে আসবে Accumulator এ, যখন $E_U = 1$ হবে এবং $\overline{L_A} = 0$ হবে।

আবার $E_A = 1$ হবে এবং $\overline{L_O} = 0$ হবে তখন এই Result তা W-Bus এ first এ আসবে, যেখান থেকে Output Register এ আসবে এবং directly Binary Display তে result তা show করবে।

Accumulator Architecture

কোন কোন register ব্যবহার করা হচ্ছে

- ① Accumulator
- ② Out Register
- ③ B
- ④ Memory Address " (MAR)
- ⑤ Instruction " (IR)

$\overline{L_A} = 0$ হয় Accumulator এ ঢুকে

$\overline{E_A} = 1$ " " " (থেকে W-BUS এ যায়।

$\overline{L_A}$ ← বাই থাকলে active করতে 0 দিতে হয়।

$\overline{L_A}$ বাই না থাকলে 1 দিতে হয়

$\overline{L_B} = 0$ দিলে Register B তে store হয়

Write two instruction which is complete by himself. $\Rightarrow$ HLT, OUT

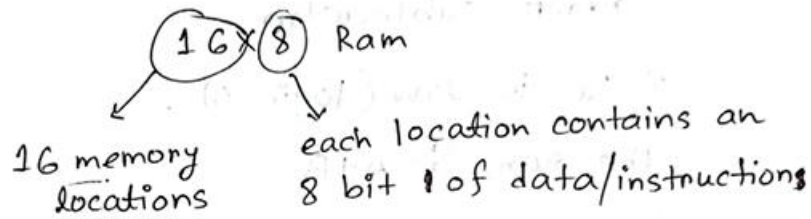
Input and MAR:

MAR stores the address of data and instruction which are place in memory.

যখন SAP-I is running-mode, the address is generated by the Program Counter which is then stored into the MAR through the W bus.

কিছুক্ষণ পর MAR থেকে 4-bit address টা RAM এ apply করে।

The RAM:



RAM 4 bit addresses MAR receives
କଲେ ଏହା read operation ଏହାକୁ performed
ହୁଏ ।

Instruction Register:

When the instruction is placed at W bus from memory, the instruction Register stores this instruction on the next positive clock edge.

- ① The upper nibble → Controller sequences
- ② The lower nibble → is read onto the W bus when needed.

Adder/Subtractor:

- 1) SAP-1 uses a 2's complement adder subtractor.
- 2) S_u is low (logic 0),
the sum, $S = A + B$
- 3) S_u is high (logic 1),
the sum, $S = A + B' + 1$
- 4) When E_u is high, those contents appear on the W bus.

Accumulator:

- i) two outputs.
 - i. One output goes to the adder/subtractor.
 - ii. Other goes to the W

- 2) when L_A is low
stores the output of adder/subtractor
- 3) when E_A is high
It's value appears on W bus

B Register :

- 1) To add/sub two 8 bit numbers A and B,
the B register stores the number B.
- 2) Supplies the number to be added
or subtracted.
- 3) When data is available at W bus
and L_b goes low, B register loads
that data.

Output Register:

- 1) arithmetic operation બાદ
accumulator contains the word
representing the answer.
- 2) Accumulator 9 ans નો stored થયે
ત્યારે W bus તરફ માધ્યમ output register
9 load કરે છે ।
- 3) This is done in the next positive
clock edge when EA is high and
Lo is low.

Controller Sequencer:

Before each computer run,
a $\overline{\text{CLR}}$ signal is sent to the PC
and a CLR " to the IR

12 wires carrying the control word are called the Control Bus.

Control word:

CON = C_p E_p $\bar{L}_m$ $\bar{C}_E$ $\bar{L}_1$ $\bar{E}_1$ $\bar{L}_A$ E_A S_v E_v $\bar{L}_B$ $\bar{L}_0$

① LDA Instruction:

LDA = load the accumulator

LDA 8H = load the accumulator with the contents of memory location 8H.

② ADD Instruction:

ADD 9H = add the data of memory location 9H with data of accumulator and save the result in accumulator

$ACC = 0000\ 0010 \rightarrow A$

$RAM[9] = 0000\ 0011 \rightarrow B$

ADD 9H

- ① RAM address 9H is loaded into the B register

$B = 0000\ 0011$

② $SUM = A + B = 0000\ 0101$

- ③ SUM is loaded into the accumulator to get

$ACC = 0000\ 0101$

③ SUB Instruction:

SUB 9H = subtract the data of memory location 9H from the data of accumulator and save the result in accumulator.

$$ACC = 0000\ 0011 \rightarrow A$$

$$RAM[9] = 0000\ 0010 \rightarrow B$$

$$Diff = A - B$$

$$\therefore ACC = 0000\ 0001$$

④ Out Instruction:

OUT tells the SAP-1 computer to transfer the accumulator contents to the output port.

Why ~~OUT~~ OUT is complete by itself?

→ Because I do not have to include an address when using OUT because the instruction does not involve data in the memory.

⑤ HLT instruction:

HLT tells the computer to stop processing data so it stops the clock.

HLT → marks the end of a programme.

④ Why HLT complete by itself?

⇒ Because I do not have to include a RAM word when using HLT because this instruction does not involve the memory.

LDA, SUB, ADD → memory-reference instructions

because they use data stored in the memory

OUT and HLT → not memory-reference instructions

because they do not involve the data stored in the memory.

Mnemonics:

Abbreviated instructions like LDA, ADD, SUB, OUT and HLT are called mneumonics.

SAP-1 OP CODES

Mnemonics	Op Code
LDA	0000
ADD	0001
SUB	0010
OUT	1110
HLT	1111

Because this code tells the computer which operation to perform, it is called an operation code. (OP Code)

❑ Why the SAP-1 instructions are upward compatible with the 8080/8085 instruction set?

⇒ Because the SAP-1 instructions LDA, ADD, SUB, OUT and HLT are 8080/8085 instructions, learning SAP-1 instructions is getting us ready for the 8080 and 8085 two widely used microprocessors.

$T_1 = 000001$
 $T_2 = 000010$
 $T_3 = 000100$

} fetch cycle

$T_4 = 001000$
 $T_5 = 010000$
 $T_6 = 100000$

} execution cycle

Ex T1-T6 Execution cycle দেখাও,
explain করে প্রত্যেকটা phase state.

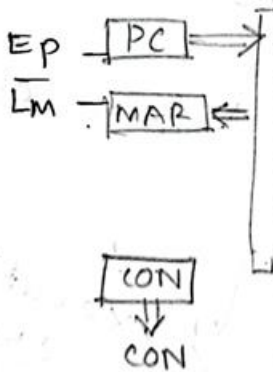
⇒ যেগুলো active সেগুলো active
দেখাবো state এ।

state. → Explain → CON

① T1 state :

address state

E_p and $\overline{L}_M$ are active

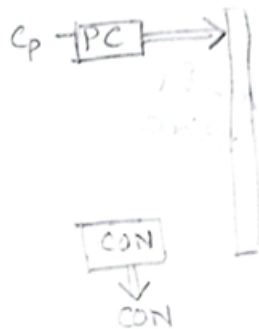


$$\begin{aligned}
 \text{CON} &= C_p \cdot E_p \cdot \overline{L}_M \cdot \overline{C_E} \cdot \overline{L}_J \cdot \overline{E}_I \cdot \overline{L}_A \cdot E_A \cdot S_U \cdot E_U \cdot \overline{L}_B \cdot \overline{L}_B \\
 &= 0 \ 1 \ 0 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 1 \ 1 \\
 &= \quad \quad 5 \quad \quad \quad E \quad \quad \quad 3
 \end{aligned}$$

② T₂ state:

increment state

C_p active



$$\begin{aligned}
 \text{CON} &= C_p \cdot E_p \cdot \overline{L_M} \cdot \overline{C_E} \cdot \overline{L_I} \cdot \overline{E_I} \cdot \overline{L_A} \cdot E_A \cdot S_U \cdot E_U \cdot \overline{L_B} \cdot \overline{L_O} \\
 &= 1 \ 0 \ 1 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 1 \ 1 \\
 &\quad \quad \quad E \quad \quad \quad 3 \\
 &= B
 \end{aligned}$$

③ T₃ state:

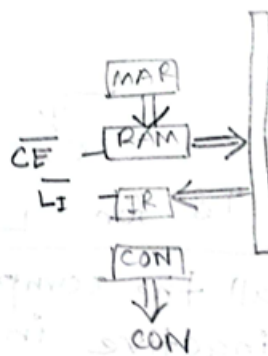
memory state

$\overline{C_E}$, $\overline{L_I}$ active

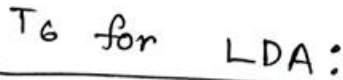
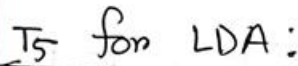
$$\text{CON} =$$

$$= 0010 \ 0110 \ 0011$$

$$= 2 \ 6 \ 3$$

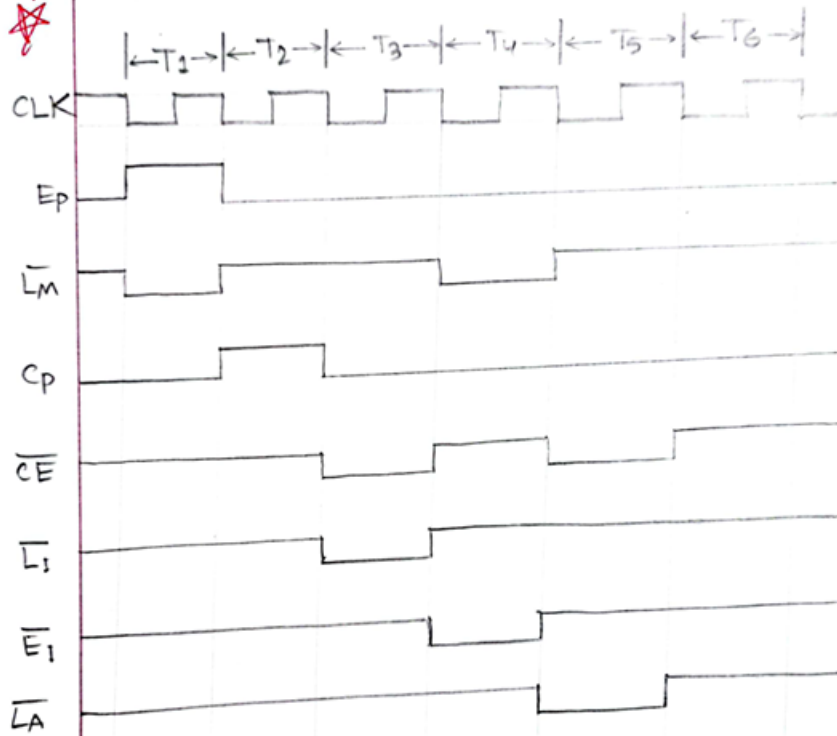


I_m, E_i
active



all the components of SAP-I will be inactive in this state. Only the CON is active.

Timing Diagram for LDA



$T_1 \rightarrow E_p, \overline{L}_m$
 $T_2 \rightarrow C_p$
 $T_3 \rightarrow \overline{C}_E, \overline{L}_i$
 $T_4 \rightarrow \overline{L}_m, \overline{E}_i$
 $T_5 \rightarrow \overline{L}_A, \overline{C}_E$
 $T_6 \rightarrow$